# DoxGPT Cost-Estimate Pipeline — AI Reference

*For AI assistants helping solo practitioners adapt the insurance-verification and patient-cost-estimate workflow to their own EMR and clearinghouse stack.*

## System Context

This pipeline turns a stack of insured referrals into per-patient cost estimates in three phases:

1. **Extract** — pull patient demographics from the EMR into a CSV in the format the clearinghouse expects for batch eligibility verification.
2. **Verify** — upload the CSV to the clearinghouse, run a batch eligibility check, download the structured response CSV.
3. **Estimate** — feed each response row into a HIPAA-covered LLM (DoxGPT) with a prompt that combines the practice's fee schedule, planned-procedure bundle, and calculation rules to produce a cost estimate.

The reference implementation uses **ModMed EMA** (Angular + PrimeNG) as the EMR, **Availity Essentials** as the clearinghouse, and **DoxGPT** as the LLM. All three are HIPAA-covered with signed BAAs.

## HIPAA Constraint

PHI moves through three vendors. Each must hold a BAA covering the practice's use. The browser-side extraction script in Phase 1 runs locally and transmits nothing — that is intentional and non-negotiable for the architecture. The CSV is written to the user's local Downloads folder. The first PHI transmission outside the user's machine is the Availity upload.

Pasting Availity output into a generic ChatGPT, Claude, Gemini, or Grok account breaks the chain. The prompt in Phase 3 is designed for DoxGPT specifically because Doximity holds a BAA covering individual physician use.

## Key DOM Selectors (ModMed Referrals / Task List)

| Element | Selector |
|---|---|
| Data table | `.p-datatable-table` |
| Header cells | `.p-datatable-table thead th` |
| Body rows | `.p-datatable-table tbody tr` |
| Pagination dropdown | `select.pagination-select` |
| Patient name link | `a.header-patient-link` (do NOT click — opens chart) |

## Column-Mapping Strategy

ModMed referral views can be customized per-practice, so column order varies. The extraction script does not hard-code column indices. Instead it reads the header row, normalizes header text, and matches on substring:

- `name` -> first column whose header contains "name"
- `dob` -> column whose header contains "dob" or "birth"
- `member` -> column whose header contains "member" or "insurance id"
- `payer` -> column whose header contains "payer" or "insurance"

This makes the script resilient to user reordering of columns within the referral view.

## Critical Implementation Notes

**Angular change detection (Phase 1):** Not relevant for the extractor — it only reads. It IS relevant for any future write-back script (e.g. posting verification results back into ModMed). For those, use `Object.getOwnPropertyDescriptor(window.HTMLInputElement.prototype, 'value').set.call(input, newValue)` then dispatch `input` and `change` events with `bubbles: true`.

**Pagination resets (ModMed):** Any navigation back to the task list via `history.back()` resets pagination to 10. Re-set `select.pagination-select` to its max value after every back-navigation if iterating across rows.

**Name parsing:** ModMed renders names as `Last, First M` in the table. Split on the first comma — not on whitespace, because some last names contain spaces (e.g. "Van Buren").

**Empty rows in tables:** PrimeNG sometimes renders a placeholder row when filters are applied. Skip rows where `cells.length < 2`.

**CSV escaping:** Wrap any field containing a comma or quote in double quotes and double any internal quotes. Names like `O'Brien, Mary` and free-text payer names like `BCBS, BlueChoice PPO` will hit this.

**Availity batch CSV column order:** Availity is strict about column order on intake. The canonical Availity Essentials batch eligibility header is: `last_name, first_name, dob, member_id, payer, provider_npi`. Other clearinghouses (Office Ally, Waystar, Change Healthcare relay) use different headers — check the documentation for the specific batch endpoint.

**Availity response — Pending rows:** Availity returns Pending when the payer has not responded yet. Pending rows have null values for deductible, OOP, copay, coinsurance. The DoxGPT prompt is built to recognize null fields and refuse to fabricate a number — it will flag the patient for re-check instead. Do not pre-fill nulls with zeros.

**Availity response — codes vs. text:** Some Availity configurations return enumeration values as text (`Active Coverage`, `Inactive`, `Pending`) and some return them as codes (`AC`, `IN`, `PE`). The DoxGPT prompt is written for text. If your Availity returns codes, translate before pasting into DoxGPT.

## DoxGPT Prompt Architecture

The prompt has five structural elements that matter more than they appear:

1. **Fee schedule in the prompt body** — not in DoxGPT's memory. Every run uses the exact schedule pasted. No drift between what the model "knows" and what the practice's contracts actually say.
2. **Medicare fallback for uncontracted payers** — prevents hallucinated allowables. Medicare allowables are public; safe default.
3. **Worst-case planned bundle with refund posture** — single number out, refund what is not performed. Avoids under-collection on visits that grow in scope.
4. **Mutually-exclusive procedures clause** — prevents double-counting CPTs that cannot both bill at the same visit. Without this clause, estimates inflate by the price of the cheaper of any conflicting pair.
5. **Flag rules for Auth Required and Inactive coverage** — the model is instructed to flag for staff review rather than produce a confident dollar amount when the data does not support one. This is the difference between a usable cost estimator and a hallucinating one.

When adapting the prompt to a different specialty, the practice changes the fee schedule and the planned bundle. The five structural elements above stay as written.

## Workflow Sequence

1. User opens EMR referrals page, sets pagination, opens Chrome DevTools console.
2. User pastes extraction script. Script reads the visible table, writes `availity_batch.csv` locally.
3. User uploads CSV to clearinghouse batch eligibility endpoint, waits 3-5 minutes, downloads response CSV.

4. User opens DoxGPT, pastes prompt template (customized once for their practice), then pastes Availity response rows.
5. DoxGPT returns structured cost estimates per patient.

## Adapting This Pattern to a Different Stack

If your stack is not ModMed + Availity + DoxGPT, paste this entire reference into Claude, Grok, or another modern AI along with three pieces of information:

1. **The name of your EMR** (e.g. Athenahealth, eClinicalWorks, Epic Hyperspace, Practice Fusion, DrChrono) and what its referrals or new-patient task page looks like in the browser. The AI will need to identify the equivalent DOM selectors. For a quick start, right-click any row in your EMR referral view, choose Inspect, and copy the outer HTML of one row into the chat.
2. **The name of your clearinghouse** (e.g. Office Ally, Waystar, Change Healthcare, pVerify, Trizetto) and whether it supports batch eligibility upload. If it does, the AI needs your clearinghouse's batch CSV header format — copy it from their documentation.
3. **Which HIPAA-covered LLM you use.** DoxGPT (Doximity) is the most common for individual physicians because the BAA covers personal use. Other covered options include Google Vertex AI with a signed BAA, Anthropic Claude via Bedrock with a BAA, or OpenAI's Enterprise tier with an executed BAA. Generic personal accounts on those same services are NOT covered.

Ask the AI to adapt the same three-phase architecture — extract from EMR, batch-verify through clearinghouse, estimate via HIPAA-covered LLM — to your specific stack. The AI now has a working reference implementation to mimic rather than inventing the workflow from scratch.

The architecture is the unlock. The specific code is the easy part once the architecture is clear.

---

*Companion to: Cost-Estimate Tutorial (this bundle). Course module: Module 5 — Insurance Verification & Patient Cost Estimates.*